Conjunto de Instrucciones

Control de Programa

CMP

- Realiza una resta entre los operandos sin modificarlos.
- Actualiza el estado del registro de banderas.

CMP EAX, EBX realiza la operación EAX – EBX

CMP CX, 56H realiza la operación CX – 56H

EFLAGS

31-18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reservados	VM	RF		NT	IO PL	IO PL	O	D	I	T	S	Z		A		P		C

Figura 1: Registro de banderas modificado por la instrucción CMP. Los bits que se modifican son el bit de Sobreflujo, el bit de Signo, el bit Cero, el Acarreo Auxiliar, el bit de Paridad, y el bit de Acarreo.

Control de Programa

- Las instrucciones de control de programa permiten que el flujo de ejecución del programa cambie.

- Esos cambios en el flujo ocurren después de decisiones hechas con las instrucciones CMP o TEST seguidas por una instrucción de salto condicional, o por medio de una instrucción de salto incondicional.

Salto incondicional **JMP**

```
        MOV AX, 1A7Fh
        MOV CL, 2
nxt:    ROL AX, CL
        INC AX
        JMP .nxt
        ADD BX, 8  (Esta instrucción nunca se ejecuta)
```

Salto incondicional **JMP**

```
      MOV AX, 1A7Fh
      MOV CL, 2
nxt:  ROL AX, CL
      INC AX
      JMP .nxt
      ADD BX, 8
```

```
073F:0111 B87F1A        MOV     AX,1A7F
073F:0114 B102          MOV     CL,02
073F:0116 D3C0          ROL     AX,CL
073F:0118 40            INC     AX
073F:0119 EBFB          JMP     0116
073F:011B 83C308        ADD     BX,+08
```

Figura 2: Resultado de ensamblar el código

Dirección instrucción MOV AX, 1A7F:	073F:0111
Dirección instrucción MOV CL, 02:	073F:0114
Dirección instrucción nxt:ROL AX, CL:	073F:0116
Dirección instrucción INC AX:	073F:0118
Dirección instrucción JMP 0116:	073F:0119
Dirección instrucción ADD BX, 08:	073F:011B

Saltos Condicionados

- Los saltos condicionados examinan los siguientes bits del registro de banderas:
    - signo (**S**), cero (**Z**), acarreo (**C**), paridad (**P**), y desbordamiento (**O**)

- Si la condición bajo prueba es verdadera, ocurre una bifurcación a la etiqueta asociada con la instrucción de salto.

- Si la condición es falsa, no se realiza el salto y por lo tanto se ejecuta la instrucción que le sigue.

Instrucción	Banderas que comprueba
JC	C = 1
JNC	C = 0
JZ, JE	Z = 1
JNZ, JNE	Z = 0
JS	S = 1
JNS	S = 0
JO	O = 1
JNO	O = 0
JP, JPE	P = 1
JNP, JPO	P = 0

Números sin signo

Instrucción	Banderas que comprueba	Descripción
JA, JNBE	CF = 0 y ZF = 0	Salta si es mayor
JAE, JNB, JNC	CF = 0	Salta si es mayor o igual
JB, JNAE, JC	CF = 1	Salta si es menor
JBE, JNA	CF = 1 ó ZF = 1	Salta si es menor o igual

Números con signo

Instrucción	Banderas que comprueba	Descripción
JG, JNLE	SF = OF y ZF = 0	Salta si es mayor
JGE	SF = OF	Salta si es mayor o igual
JL, JNGE	SF ≠ OF	Salta si es menor
JLE, JNG	SF ≠ OF ó ZF = 1	Salta si es menor o igual

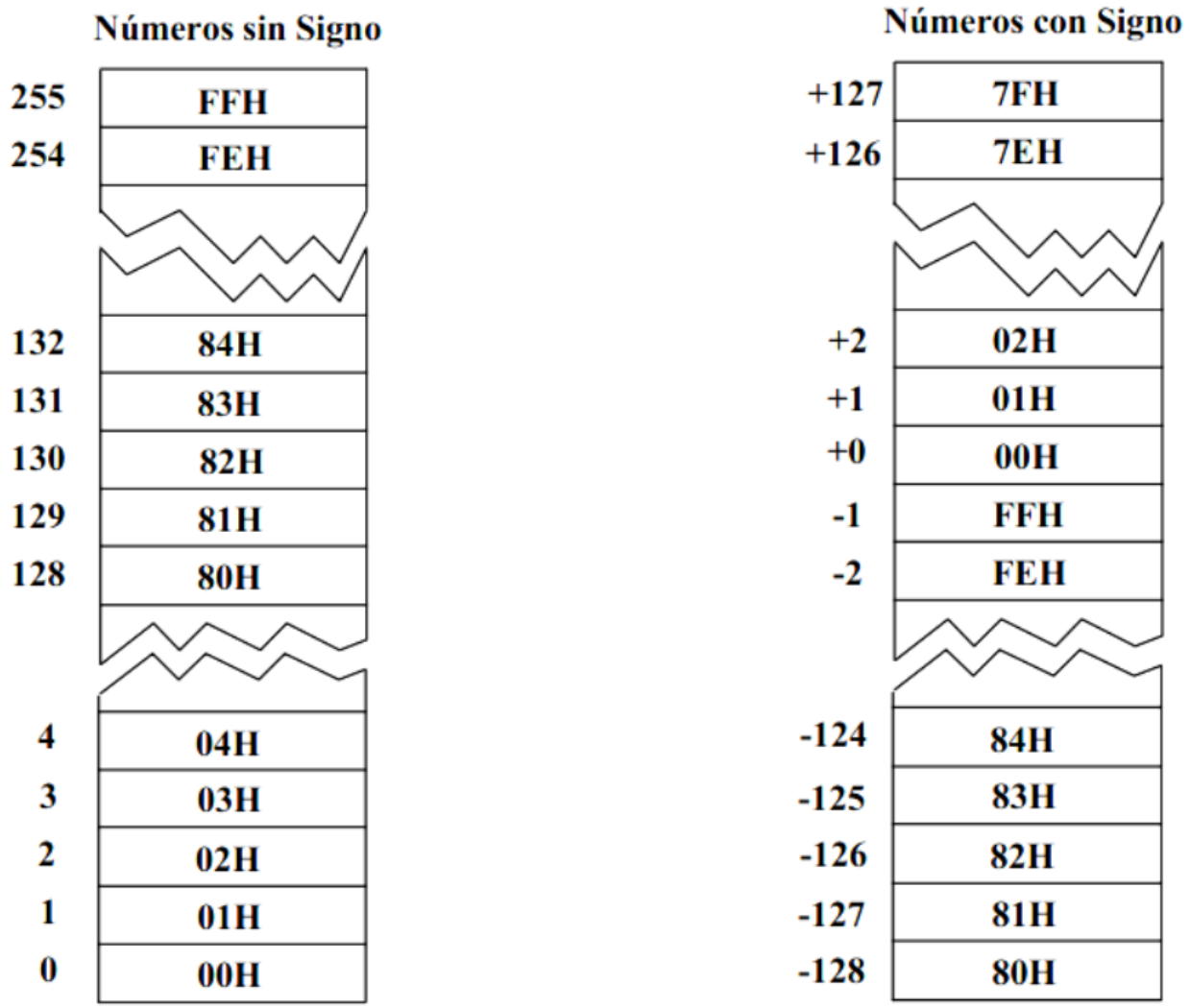

Figura 3: Comparación números con signo. Con un dato de 1 byte utilizando representado como número sin signo podemos contar desde 0 hasta 255, en cambio utilizando una representación de números sin signo, podemos contar desde -128 hasta +127. Resaltar como mismos valores hexadecimales como FFh pueden representar dos números, 255 ó -1 según el tipo de representación que se esté utilizando.

JCXZ y JECXZ

- JCXZ:

  Ocurre salto solo si CX = 0.

- JECXZ:

  Ocurre salto solo si ECX = 0

Ejemplo 1

- Programa que lee un dato de 8 bits del puerto 0x2456, si el dato es mayor o igual a 0x1F, envía un 0xAA en el puerto 0x2899, de lo contrario envía un 0xBB.

Ejemplo 1

- Programa que lee un dato de 8 bits del puerto 0x2456, si el dato es mayor o igual a 0x1F, envía un 0xAA en el puerto 0x2899, de lo contrario envía un 0xBB.

```
        MOV DX, 0x2456
        IN AL, DX
        CMP AL, 0x1F
        JAE .mayor
        MOV AL, 0xBB
        JMP .out
.mayor: MOV AL, 0xAA
.out:   MOV DX, 0x2899
        OUT DX, AL
```

Ejemplo 2

- Programa que lee un caracter del puerto 0x80, si el dato es una letra *'a'*, envía un carácter T (indicando TRUE) en el puerto 0x92, de lo contrario envía una F (indicando FALSE).

Ejemplo 2

- Programa que lee un caracter del puerto 0x80, si el dato es una letra *'a'*, envía un carácter T (indicando TRUE) en el puerto 0x92, de lo contrario envía una F (indicando FALSE).

```
        IN AL, 0x80
        CMP AL, 'a'
        JE .es_a
        MOV AL, 'F'
        JMP .out
.es_a:  MOV AL, 'T'
.out:   OUT 0x92, AL
```

Ejemplo 3

- Programa que lee un dato de 8 bits del puerto 0x2345, si el bit 3 del dato está activo, envía un carácter T (indicando TRUE) en el puerto 0x92, de lo contrario envía una F (indicando FALSE).

Ejemplo 3

- Programa que lee un dato de 8 bits del puerto 0x2345, si el bit 3 del dato está activo, envía un carácter T (indicando TRUE) en el puerto 0x92, de lo contrario envía una F (indicando FALSE).

```
        MOV DX, 0x2345
        IN AL, DX
        TEST AL, 0x8
        JNZ .es_1
        MOV AL, 'F'
        JMP .out
.es_1:  MOV AL, 'T'
.out:   OUT 0x92, AL
```

LOOP

- La instrucción LOOP es una combinación de un decremento en CX y un salto condicionado.

- Esta instrucción decrementa a CX y si CX es diferente de cero, se salta a la dirección indicada por la etiqueta.

- Si CX llega a cero, no ocurre el salto y por lo tanto se ejecuta la instrucción que le sigue.

Ejemplo

- Programa que envía los caracteres del abecedario por el puerto 0x92, uno a uno.

```
        MOV AL, 'a'
        MOV CX, 26
.out:   OUT 0x92, AL
        INC AL
        LOOP .out
```

LOOPE

- Salta si CX es diferente de cero mientras una condición de igual existe (Z=1).

- Se saldrá del ciclo si la condición no es igual o si el registro CX se decrementa a cero.

Ejemplo

- Programa que busca el primer byte diferente a cero en un arreglo de 16 elementos.

```
        MOV CX, 16    ;Arreglo de 16 elementos
        MOV BX, -1
.busca: INC BX
        CMP [Arreglo + BX], 0
        LOOPE .busca
        JE .verif_cero
```

LOOPNE

- Salta si CX es diferente de cero mientras una condición de no igual exista (Z=0).

- Se saldrá del ciclo si la condición es igual o si el registro CX se decrementa a cero.

Ejemplo

- El bit 7 del puerto 0x379 es cero si un dispositivo conectado al puerto está ocupado y uno si no lo está. Si se quieren escribir datos al puerto, se necesita verificar el estado del bit. Escriba un programa que verifique máximo 65,536 veces si el puerto está ocupado.

```
        MOV DX, 0x379
        MOV CX, 0
.espera: IN AL, DX
        TEST AL, 0x80
        LOOPNE .espera
        JNE .timedout
```

Formato de las instrucciones JMP

- Salto corto (short).
- Salto cercano (near).
- Salto lejano (far).
- Salto con registros como operando.
- Salto usando memoria.

Salto corto (short)

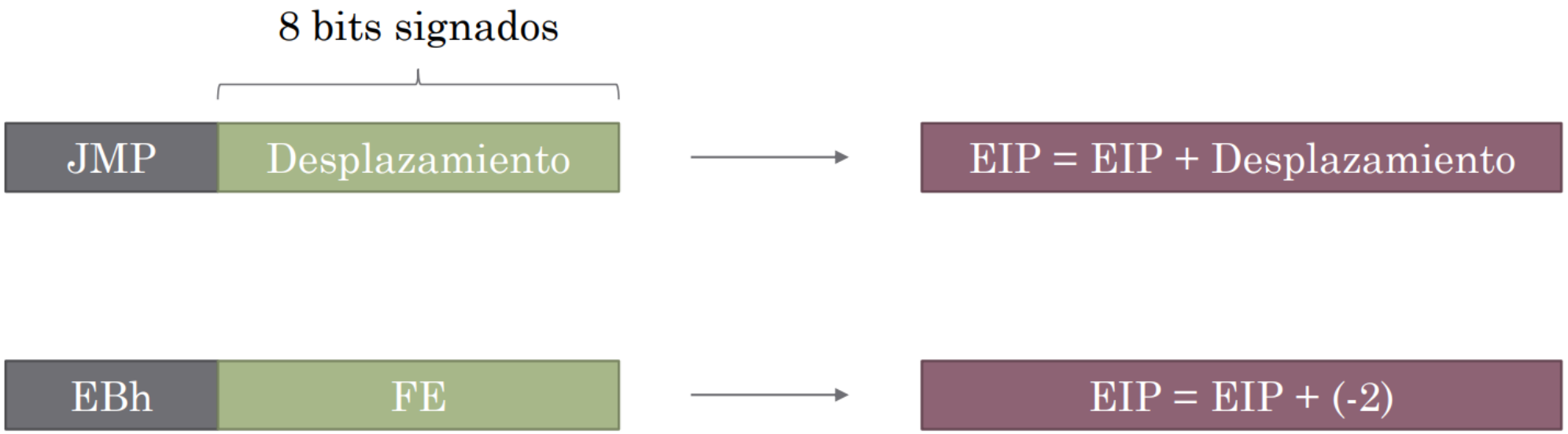

Figura 4: Salto corto. Sintaxis: JMP Desplazamiento. Desplazamiento es un número de 8 bits con signo. La instrucción suma a EIP el desplazamiento. Ejemplo: JMP FEh realiza la operación EIP = EIP + (-2).

Salto cercano (near)

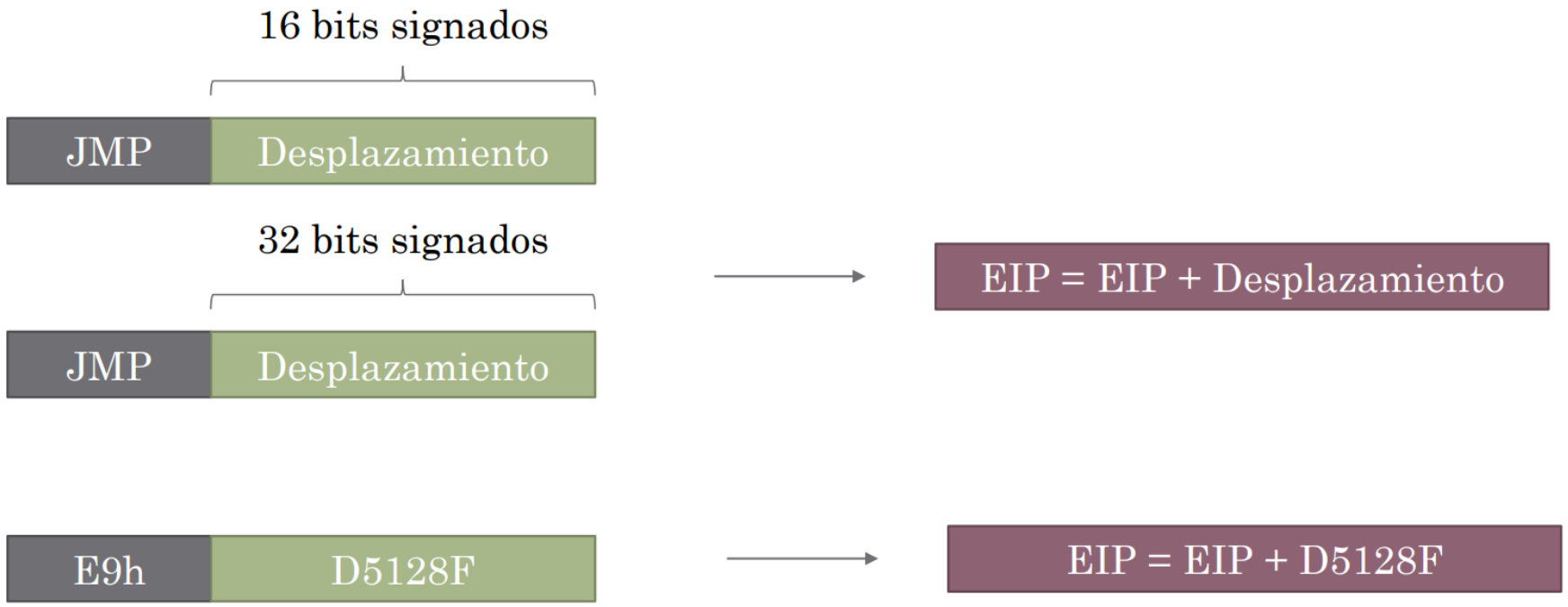

Figura 5: Salto cercano. Sintaxis: JMP Desplazamiento. Desplazamiento puede ser un número de 16 o 32 bits con signo. La instrucción suma a EIP el desplazamiento. Ejemplo: JMP D5128Fh realiza la operación EIP = EIP + D5128F.

Salto lejano (far)

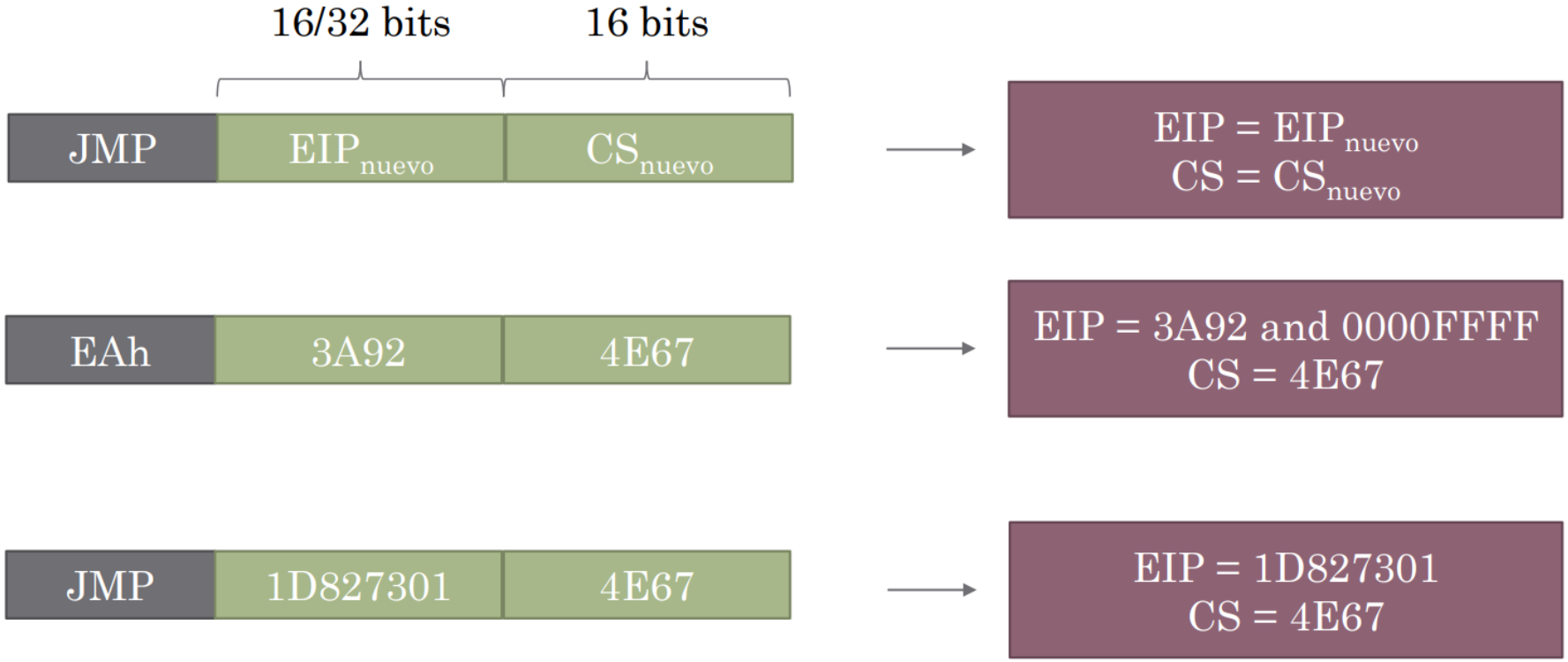

Figura 6: Salto lejano. Sintaxis: JMP Nuevo_EIP Nuevo_CS. Nuevo_EIP puede ser un número de 16 o 32 bits. Nuevo_CS es un número de 16 bits. La instrucción asigna nuevos valores a EIP y CS. Ejemplo: JMP 1D827301 4E67 realiza las asignaciones EIP = 1D827301, CS = 4E67.

Salto con registro como operando

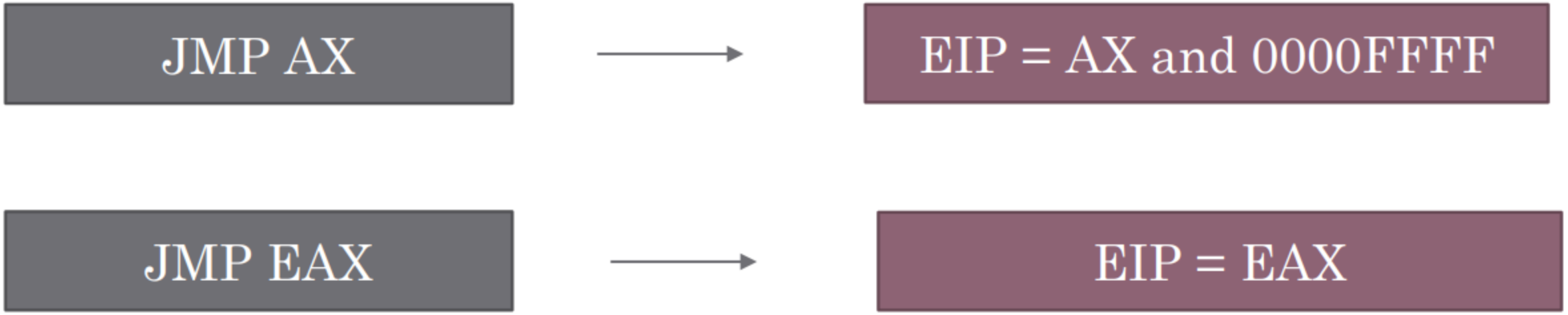

Figura 7: Salto con registro como operando. Sintaxis JMP registro. Registro puede ser de 16 o 32 bits. Ejemplo JMP AX realiza la operación EIP = AX and 0000FFFF. JMP EAX realiza la operación EIP = EAX.

Salto usando memoria

JMP near[ESI] → $EIP = Mem[ESI]_{16}$ and 0000FFFF
ó
$EIP = Mem[ESI]_{32}$

JMP far[EAX] → $EIP = Mem[EAX]_{16}$ ó $Mem[EAX]_{32}$
$CS = Mem[EAX + (2 \text{ ó } 4)]_{16}$

Figura 8: Salto a un valor almacenado en memoria. Ejemplos: JMP near [ESI] realiza la operación EIP = Mem[ESI] and 0000FFFF si el dato leído es de 16 bits ó EIP = MEM[ESI] si el dato leído es de 32 bits. JMP far[EAX] almacena los primeros 2 ó 4 bytes leídos en EIP y los siguientes 2 byes en CS.

Saltos condicionados

- Los Saltos condicionados son cortos o cercanos.

Procedimientos

- **CALL:** Realiza un salto hacia un procedimiento. Guarda en la pila la dirección de retorno

- **RET:** Realiza un salto hacia la dirección de retorno, sacándola de la pila.

```
MOV AL, 3Fh
CALL printBin
```

```
printBin:
        PUSH AX
        PUSH CX
        MOV AH, AL
        MOV CX, 8
.print: XOR AL, AL
        SHL AH, 1
        ADC AL, '0'
        CALL putchar
        LOOP .print
        POP CX
        POP AX
        RET
```

CALL y RET

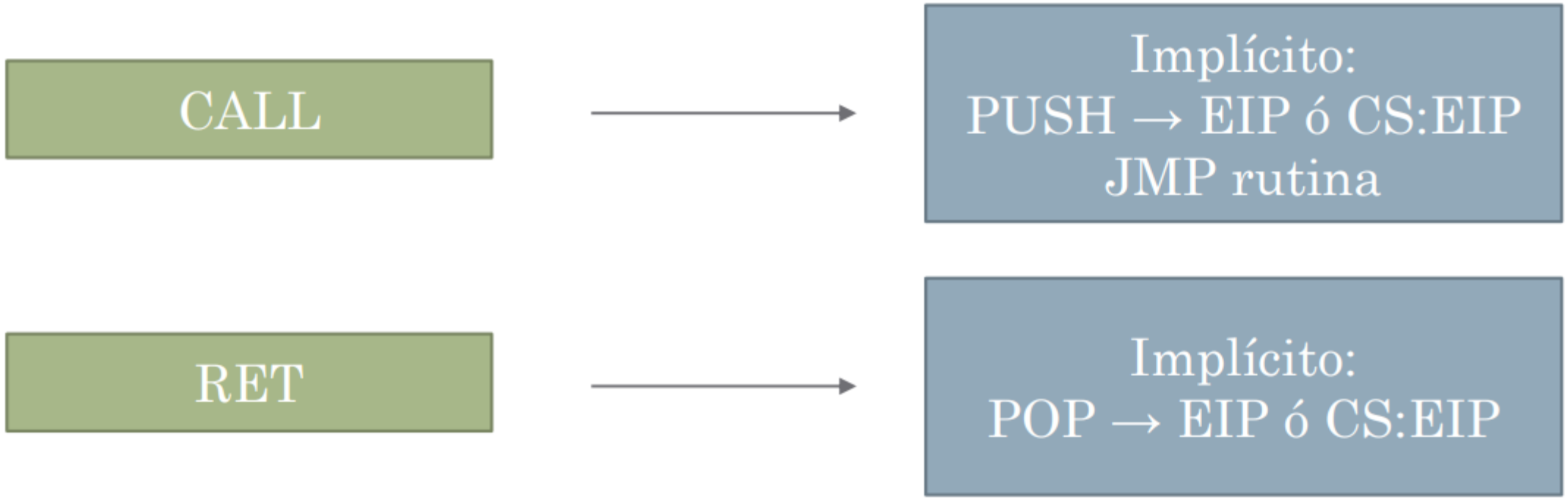

Figura 9: Instrucción CALL. De manera implícita realiza las instrucciones PUSH EIP ó PUSH CS:EIP, JMP rutina.
Instrucción RET. De manera implícita realiza las instrucciones POP EIP ó POP CS:EIP

Llamada cercana (near)

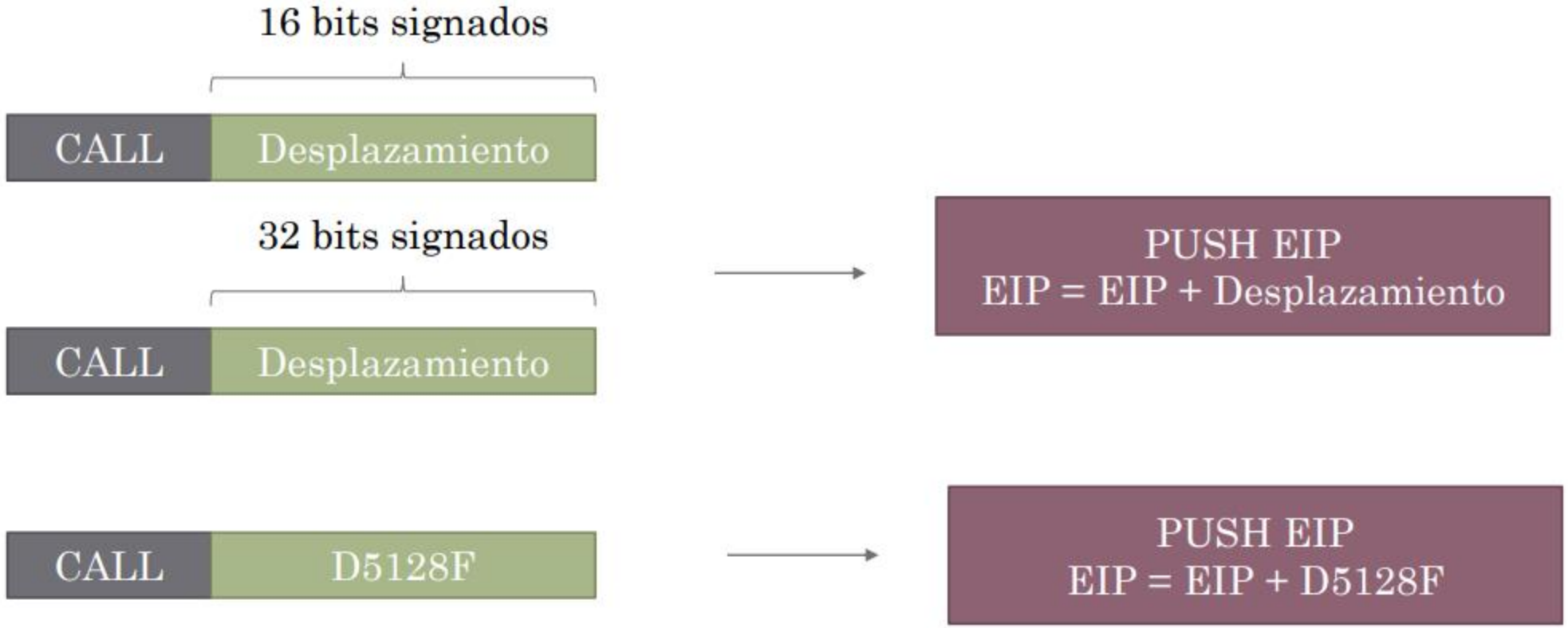

Figura 10: Llamada cercana. Sintaxis: CALL Desplazamiento. Desplazamiento puede ser un número de 16 o 32 bits con signo. La instrucción guarda EIP en la pila y suma a EIP el desplazamiento. Ejemplo: CALL D5128Fh realiza la operación PUSH EIP, seguido de, EIP = EIP + D5128F.

Llamada lejana (far)

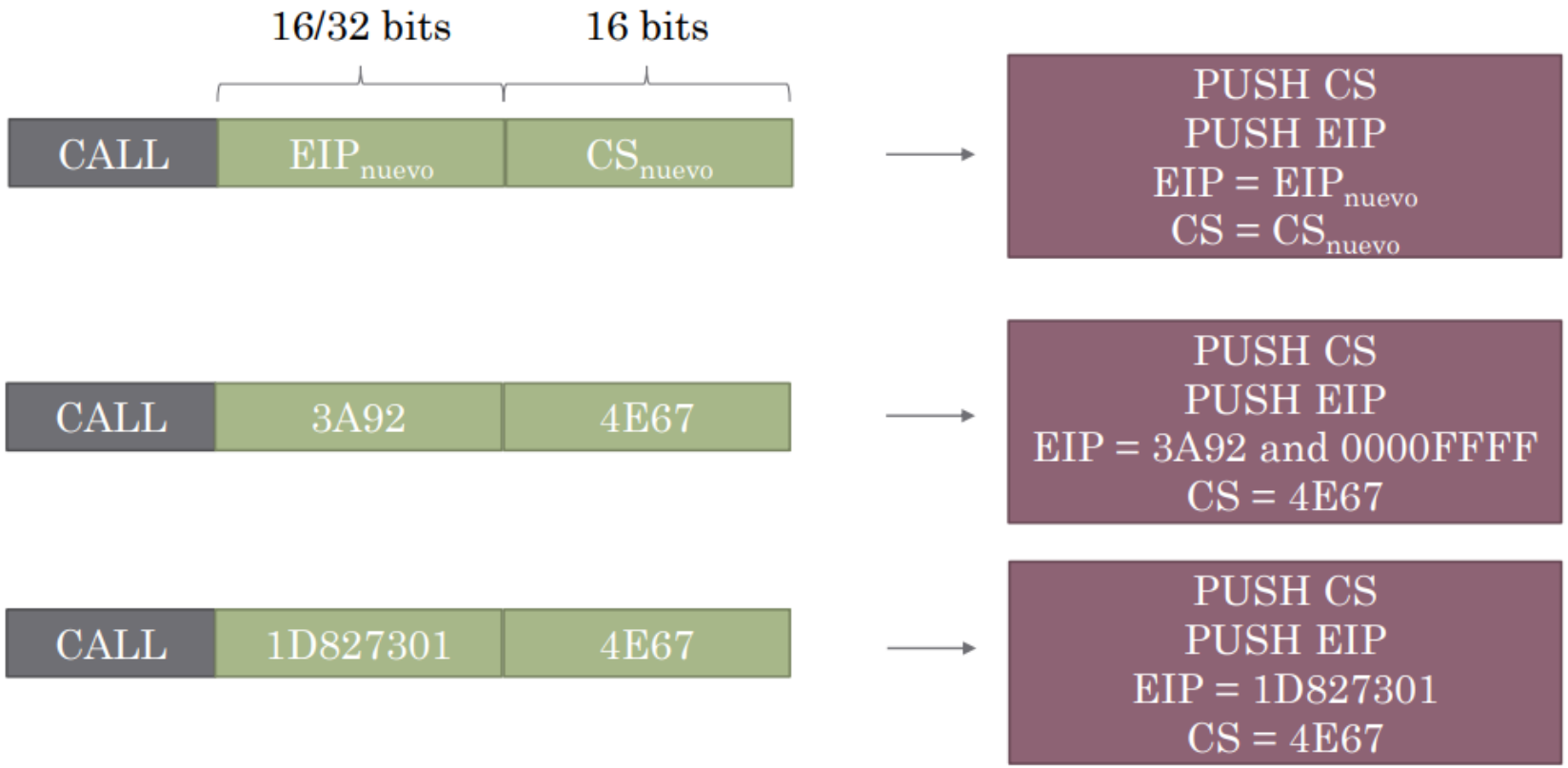

Figura 11: Llamada lejana. Sintaxis: CALL Nuevo_EIP Nuevo_CS. Nuevo_EIP puede ser un número de 16 o 32 bits. Nuevo_CS es un número de 16 bits. La instrucción guarda en la pila a CS y EIP , después, asigna nuevos valores a EIP y CS.
Ejemplo: CALL 1D827301 4E67 realiza las operaciones PUSH CS, PUSH EIP, luego realiza, EIP = 1D827301, CS = 4E67.

Llamada con registro como operando

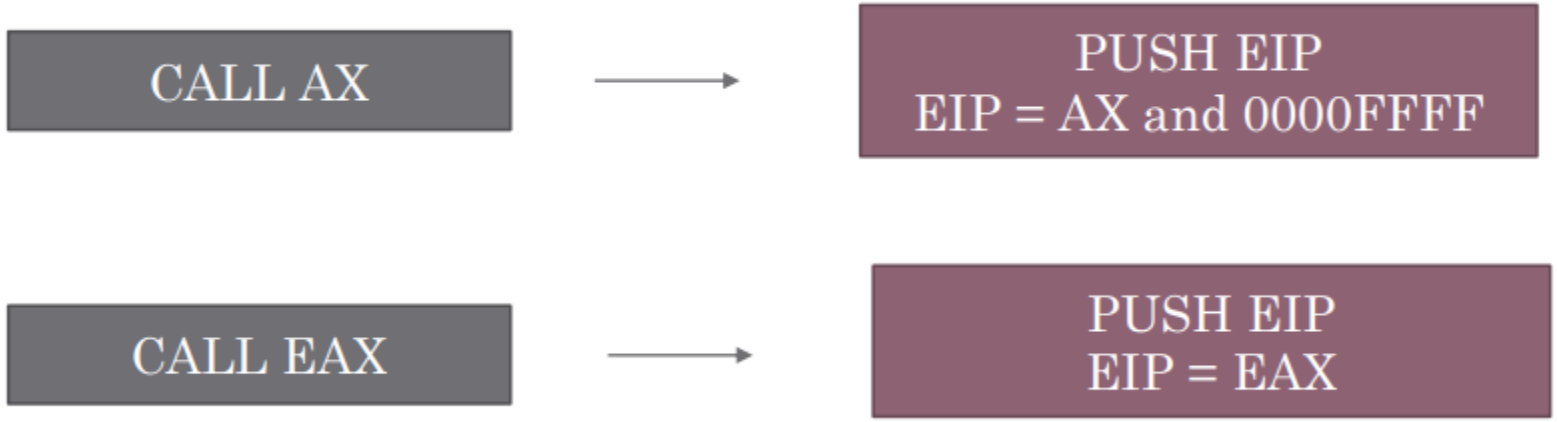

Figura 12: Llamada con registro como operando. Sintaxis CALL registro. Registro puede ser de 16 o 32 bits. Ejemplo CALL AX realiza la operación PUSH EIP, y EIP = AX and 0000FFFF. JMP EAX realiza la operación PUSH EIP seguida de EIP = EAX.

Llamada usando memoria

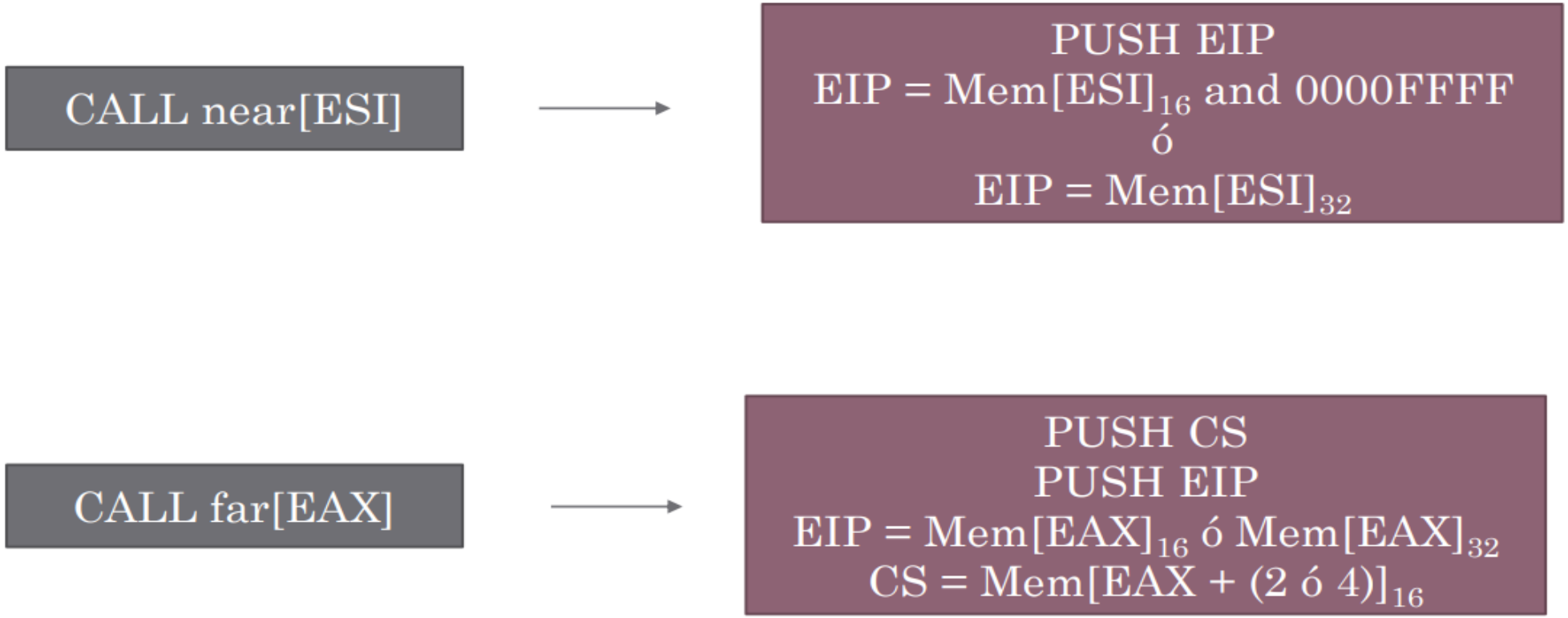

Figura 13: Llamada a un valor almacenado en memoria. Ejemplos: CALL near [ESI] realiza la operación PUSH EIP, seguida de, EIP = Mem[ESI] and 0000FFFF si el dato leído es de 16 bits ó EIP = MEM[ESI] si el dato leído es de 32 bits.
CALL far[EAX] mete a la pila CS y EIP, luego, almacena los primeros 2 ó 4 bytes leídos en EIP y los siguientes 2 byes en CS.

RET

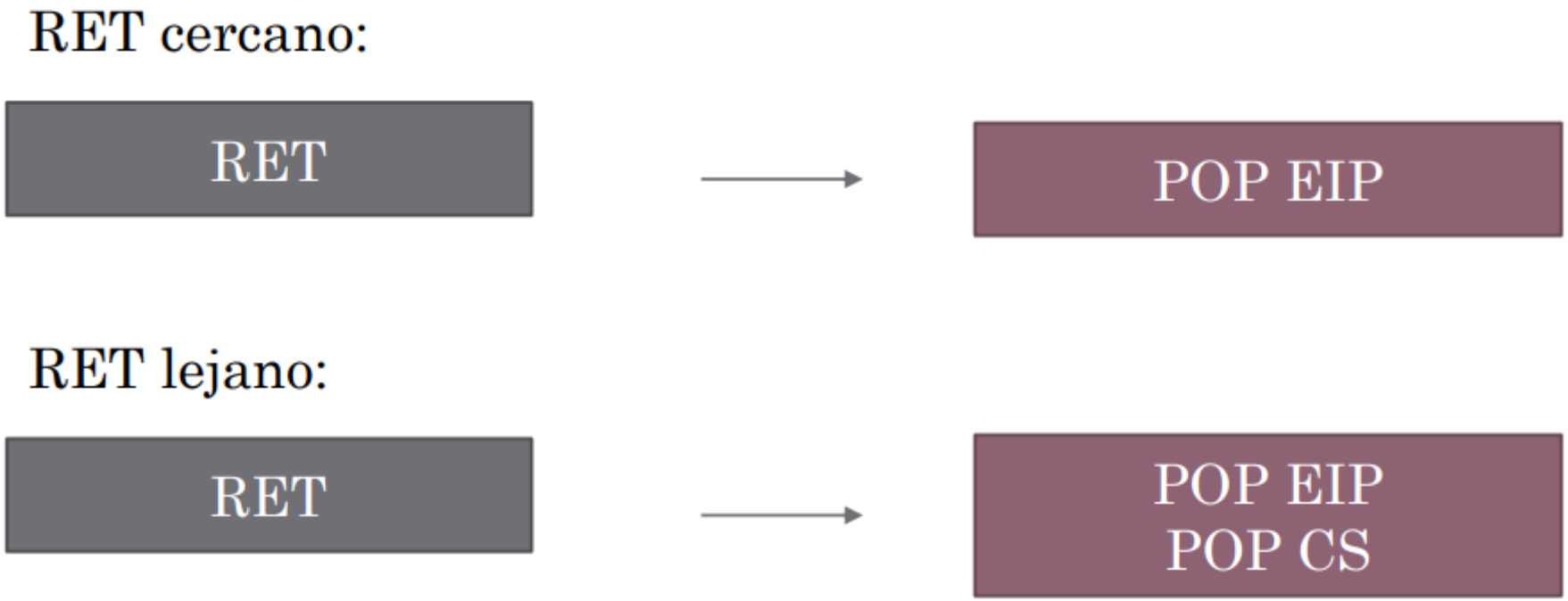

Figura 14:
RET cercano: saca un valor de la pila y lo asigna a EIP usando la instrucción POP EIP.
RET lejano: saca dos valores de la pila y los asigna a EIP y CS usando las instrucciones POP EIP y POP CS.

Instrucciones misceláneas

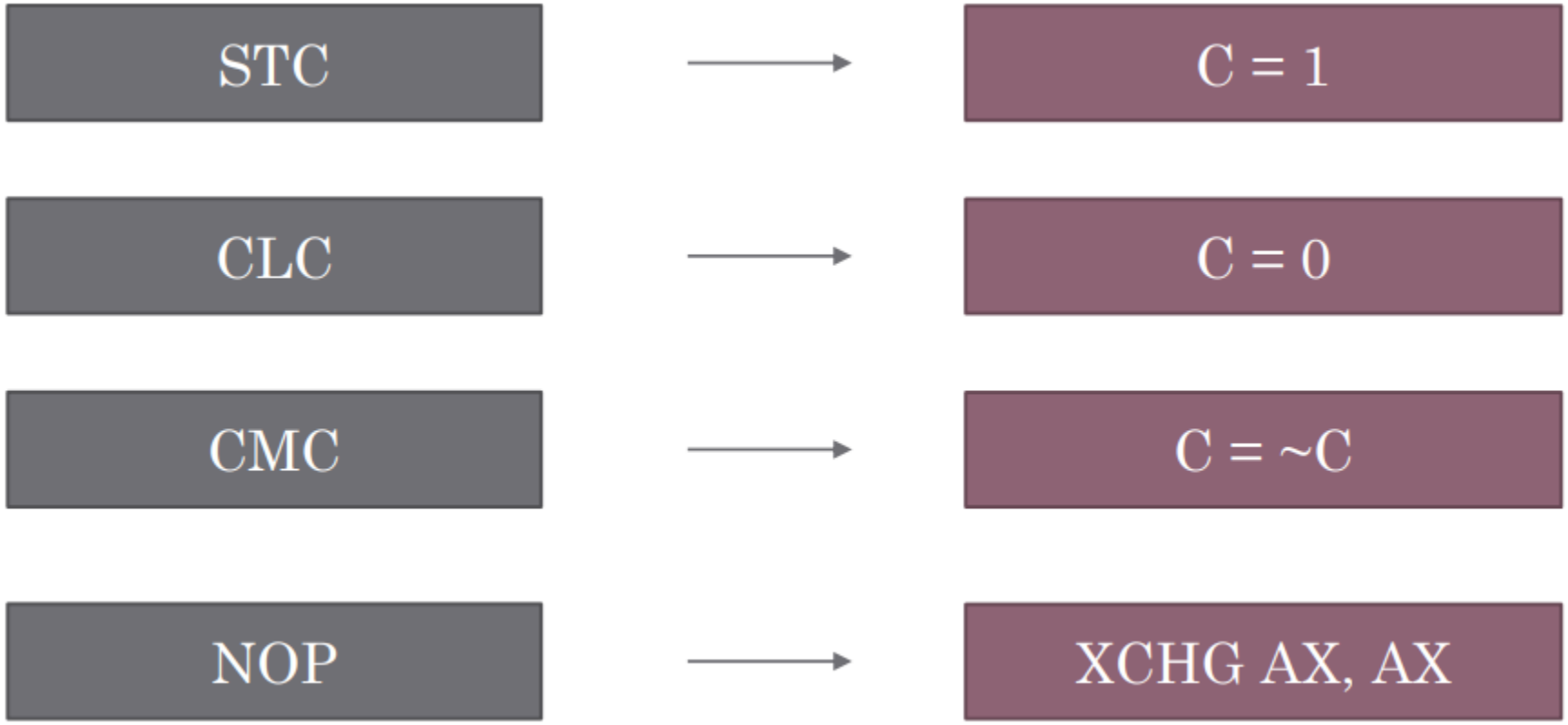

Figura 15: Otras instrucciones.
STC coloca en 1 la bandera de acarreo
CLC coloca en 0 la bandera de acarreo
CMC complementa la bandera de acarreo
NOP No realiza ninguna operación